Database Architecture Report: Bridge Platform

1. Architectural Overview

The Bridge Platform database is built on **PostgreSQL**, utilizing a microservices-oriented relational structure. The primary objective of this architecture is to decouple individual human identities from corporate entities while maintaining a strict "Bridge" between freelancers and startups. By leveraging **UUIDs** (Universally Unique Identifiers) for all primary keys and **Class Table Inheritance** for user roles, the system ensures data integrity, security, and scalability across different technological stacks (Django, FastAPI, and Spring Boot).

2. Core Service Logic

2.1 User Service (Identity & Roles)

This service acts as the foundation of the platform, managing authentication and professional personas. Unlike traditional systems where a user *is* a startup, our architecture treats the **Startup** as a standalone business entity linked to a human owner.

- **Users Table:** The central hub for authentication, storing secure password hashes, emails, and global status flags.
- **Roles Table:** A strict lookup table that enforces the user's primary permission set (Admin, Freelancer, or Startup Owner).
- **Admins & Freelancers:** These tables extend the core User table via a 1:1 inheritance pattern, allowing for role-specific attributes like professional skills for freelancers or access levels for admins without cluttering the main identity table.

2.2 Project Service (The "Bridge" Logic)

The Project Service facilitates the transactional relationship between the employer and the talent. It manages the lifecycle of a job from a public advertisement to a private work agreement.

- **Missions:** These are the primary job postings. They are owned by the Startup entity (rather than a specific user) to ensure business continuity.
- **Applications & AI Matching:** Freelancers apply to missions through a dedicated bridge table. This table stores AI match scores generated by the DeepSeek integration, allowing the platform to rank applicants based on skill relevance.
- **Contracts:** Once an application is accepted, a contract is generated. This serves as the "transactional link" between the two parties, governing the work status and deliverables.

2.3 Payment & Analytics Services

To ensure trust and transparency, the platform includes integrated financial and feedback mechanisms.

- **Escrow System:** A dedicated table holds mission funds in a "locked" state upon contract signature. These funds are only released to the freelancer's wallet upon verified completion of deliverables.
- **Wallets:** Every hiring entity and service provider has a virtual wallet to track balances and transaction history.
- **Feedback & Reputation:** The review system allows for mutual ratings after every contract, feeding data back into the freelancer's aggregate AI score to help future clients make informed decisions.

3. Detailed Data Dictionary

3.1 Identity & Profile Attributes

- **Users (`users`):**
  - `id`: Unique UUID for the account.
  - `email`: Validated unique email address.
  - `password_hash`: Encrypted credentials using industry-standard hashing.
  - `role_id`: Foreign key determining user type.
- **Freelancers (`freelancers`):**
  - `skills`: A PostgreSQL array of technical keywords used for matching.
  - `hourly_rate`: The baseline professional rate for the freelancer.
  - `ai_aggregate_score`: A calculated reputation metric (0.00 - 5.00).
- **Startups (`startups`):**
  - `owner_id`: Foreign key linking to the human user who manages the entity.

    - `startup_name`: Public-facing name of the company.
    - `mission_cnt`: An integer tracking total platform activity.

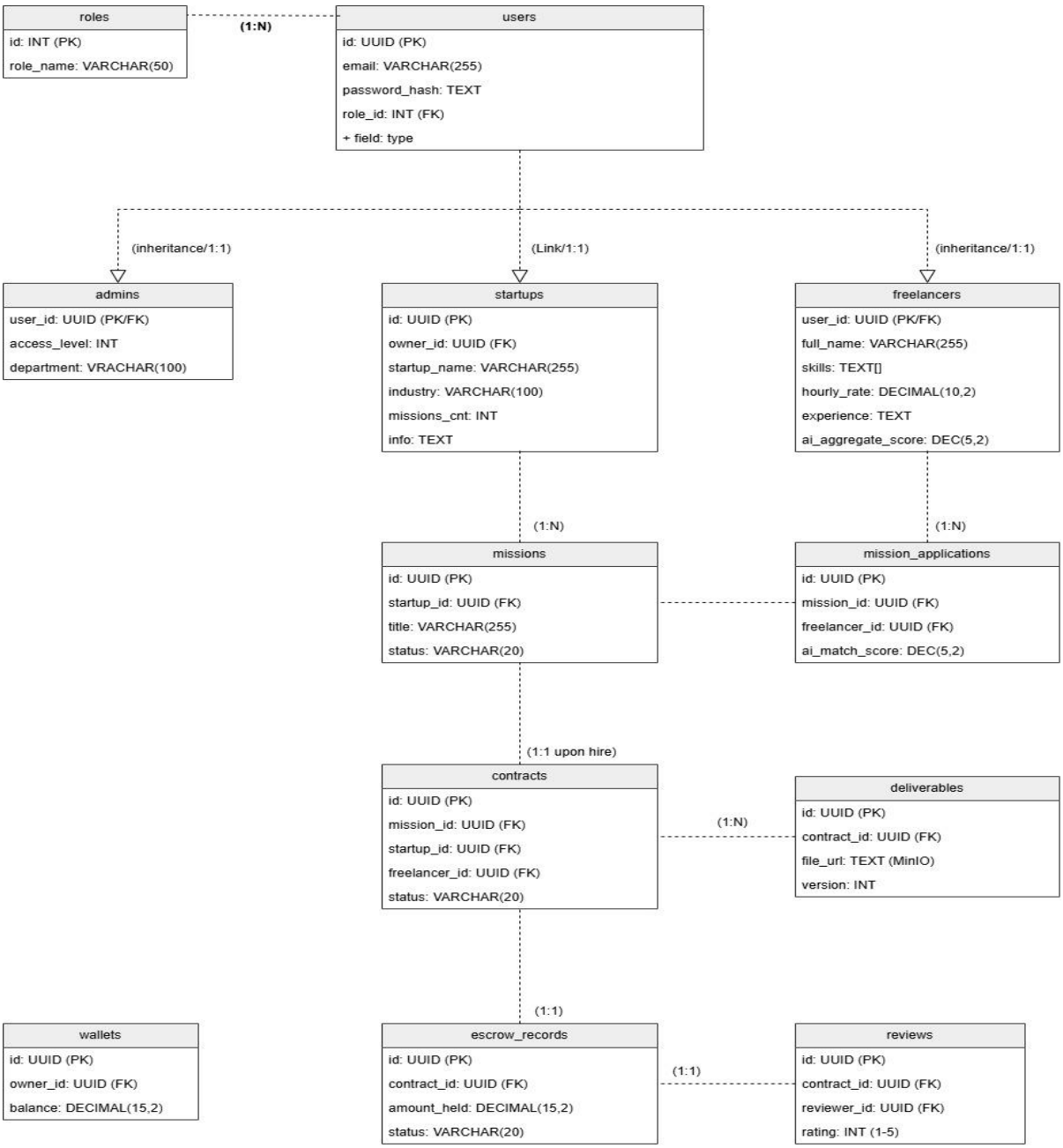
3.2 Transactional & Work Attributes

- **Missions (`missions`):**
    - `startup_id`: Links the posting to the corporate entity.
    - `title/description`: Details of the required work.
    - `status`: Lifecycle tracker (OPEN, IN_PROGRESS, CLOSED).
- **Contracts (`contracts`):**
    - `freelancer_id / startup_id`: The two parties linked by the agreement.
    - `status`: Real-time state of the project (ACTIVE, COMPLETED, DISPUTED).
- **Deliverables (`deliverables`):**
    - `file_url`: A secure reference to the MinIO/Cloud storage path for work files.
    - `version`: Tracks iterations of the submitted work.

4. Financial Safety Mechanisms

The platform utilizes an **Escrow Record** system to mitigate risk. Each record contains the specific `contract_id` and the `amount_held`. This ensures that:

- Freelancers are guaranteed payment exists before starting work.
- Startups are guaranteed that funds are only released when the contract status transitions to "COMPLETED."
- Wallets track `balance` as a liquid decimal type to prevent rounding errors during multi-currency or fee calculations.


roles
id: INT (PK)
role_name: VARCHAR(50)
(1:N)
users
id: UUID (PK)
email: VARCHAR(255)
password_hash: TEXT
role_id: INT (FK)
+ field: type
(inheritance/1:1)
(Link/1:1)
(inheritance/1:1)
admins
user_id: UUID (PK/FK)
access_level: INT
department: VRACHAR(100)
startups
id: UUID (PK)
owner_id: UUID (FK)
startup_name: VARCHAR(255)
industry: VARCHAR(100)
missions_cnt: INT
info: TEXT
freelancers
user_id: UUID (PK/FK)
full_name: VARCHAR(255)
skills: TEXT[]
hourly_rate: DECIMAL(10,2)
experience: TEXT
ai_aggregate_score: DEC(5,2)
(1:N)
(1:N)
missions
id: UUID (PK)
startup_id: UUID (FK)
title: VARCHAR(255)
status: VARCHAR(20)
mission_applications
id: UUID (PK)
mission_id: UUID (FK)
freelancer_id: UUID (FK)
ai_match_score: DEC(5,2)
(1:1 upon hire)
contracts
id: UUID (PK)
mission_id: UUID (FK)
startup_id: UUID (FK)
freelancer_id: UUID (FK)
status: VARCHAR(20)
(1:N)
deliverables
id: UUID (PK)
contract_id: UUID (FK)
file_url: TEXT (MinIO)
version: INT
(1:1)
wallets
id: UUID (PK)
owner_id: UUID (FK)
balance: DECIMAL(15,2)
escrow_records
id: UUID (PK)
contract_id: UUID (FK)
amount_held: DECIMAL(15,2)
status: VARCHAR(20)
(1:1)
reviews
id: UUID (PK)
contract_id: UUID (FK)
reviewer_id: UUID (FK)
rating: INT (1-5)